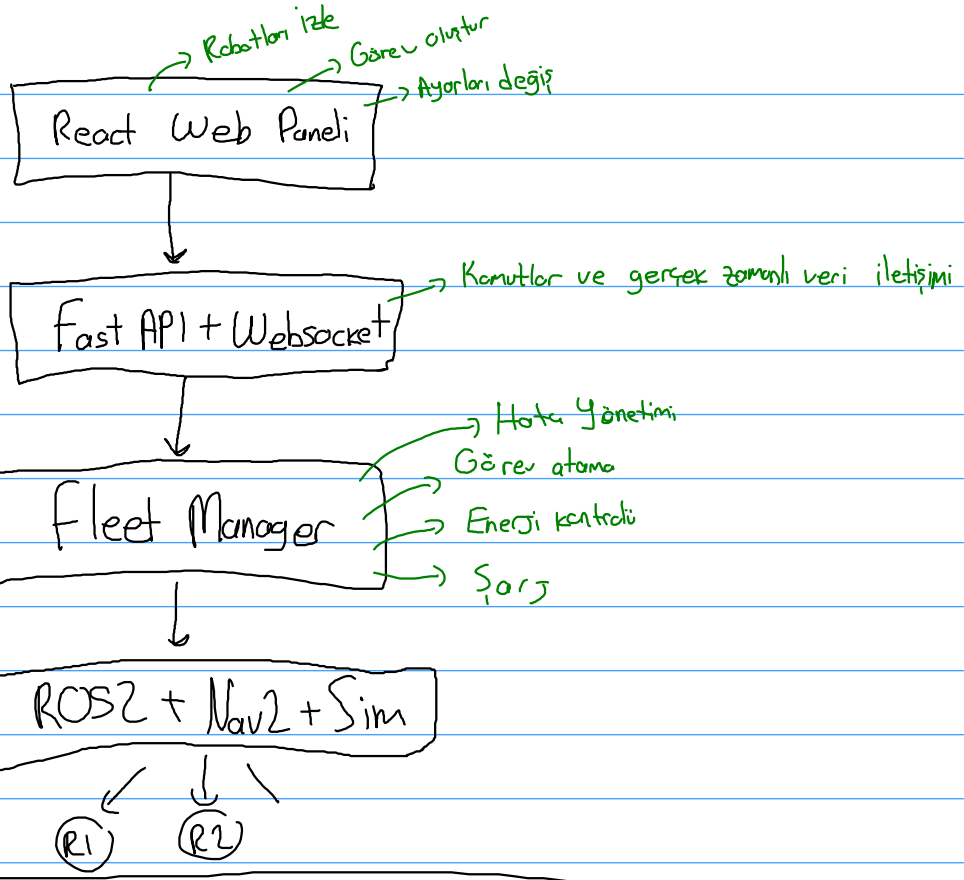
Ros2-Fleet-Manager



Projenin Ana Bileşenleri

1) **Multi-Robot Ros2/Nav2**

→ Robotların simülasyonda oluşturulmasını, haritada konumlandırılmasını ve otonom navigasyonunu sağlar

2) **Fleet Manager**

→ Görevleri sıraya alır, en uygun robotu seçer, Nav2 ye hedef gönderir, görevlerin durumunu takip eder ve hata oluştuğunda nasıl davranacağına karar verir.

3) **Battery Simulator**

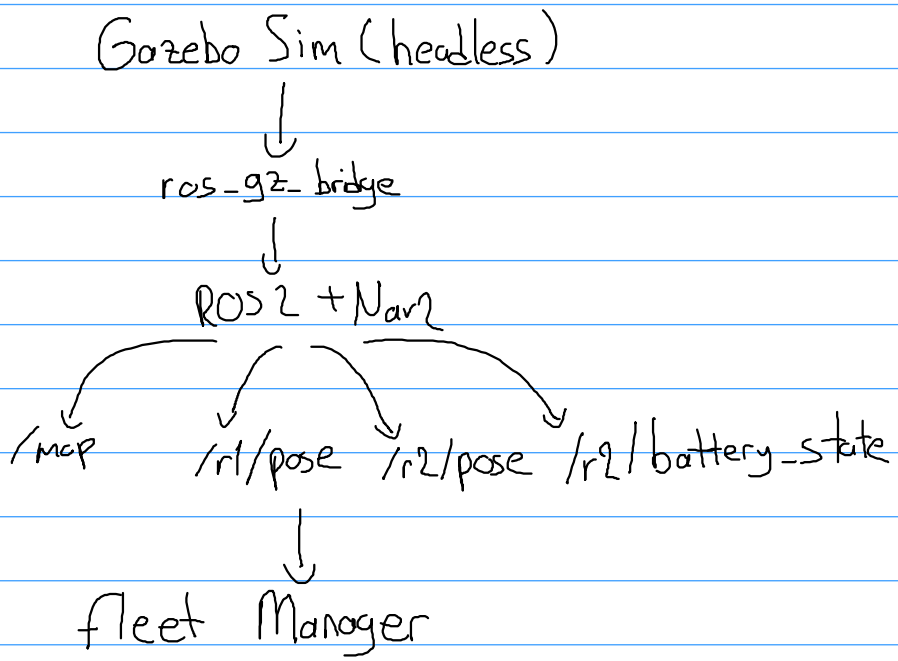
→ Robotların hareket ve bekleme sürelerine göre batarya tüketimi simüle eder. Şarj istasyonunda doğru şekilde yanaşıp temas koşullarını sağladığında karar verir.

4) **Fast API + Websocket + React**

→ Kullanıcının robotları harita üzerinde takip etmesini, görev oluşturmasını, yapılandırmayı düzenlemesini ve deney sonuçlarını incelemesini sağlar.

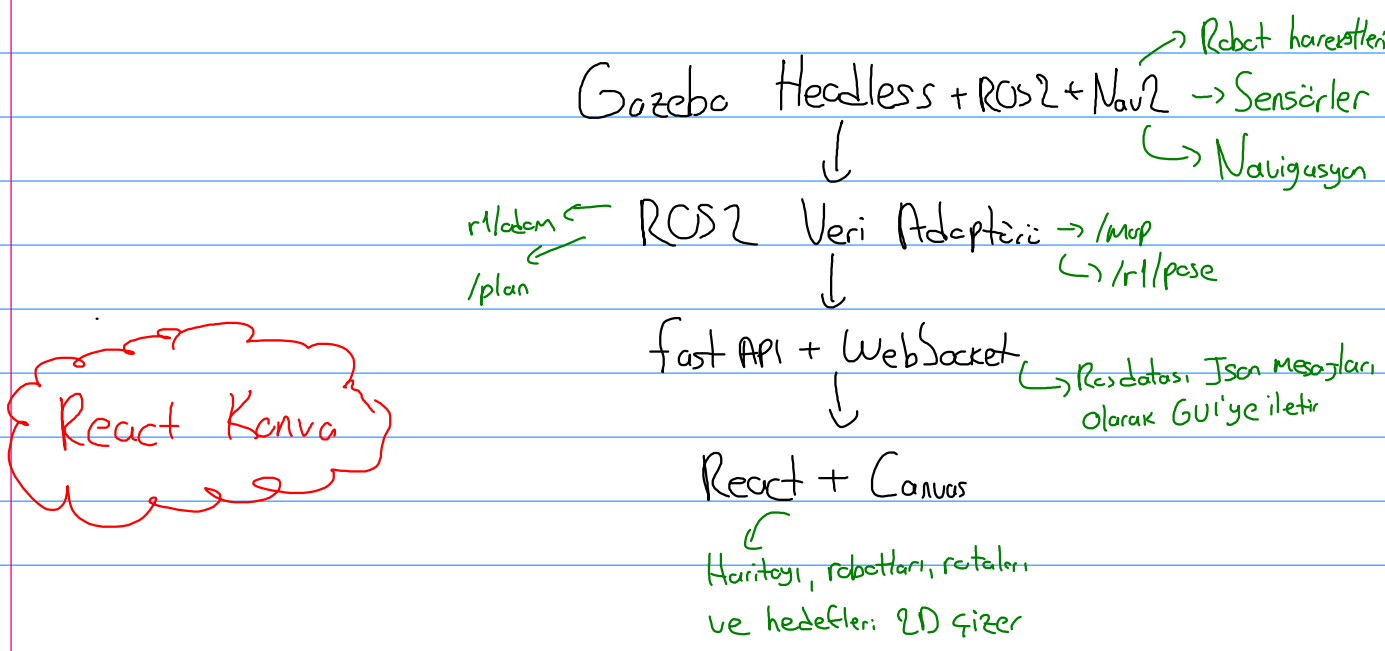
Simülasyon Mimarisi

- Gazebo arka planda headless olarak çalışır, 2D GUI üzerinde resmedilir. Gazebo simüle eder kullanıcı 2D harita görür.



- Harita düşürmek için nav_msg/OccupancyGrid kullanılacak
- Robotların konumlarını Tf üzerine çizilir.
- Planlanan yollar nav_msgs/Path

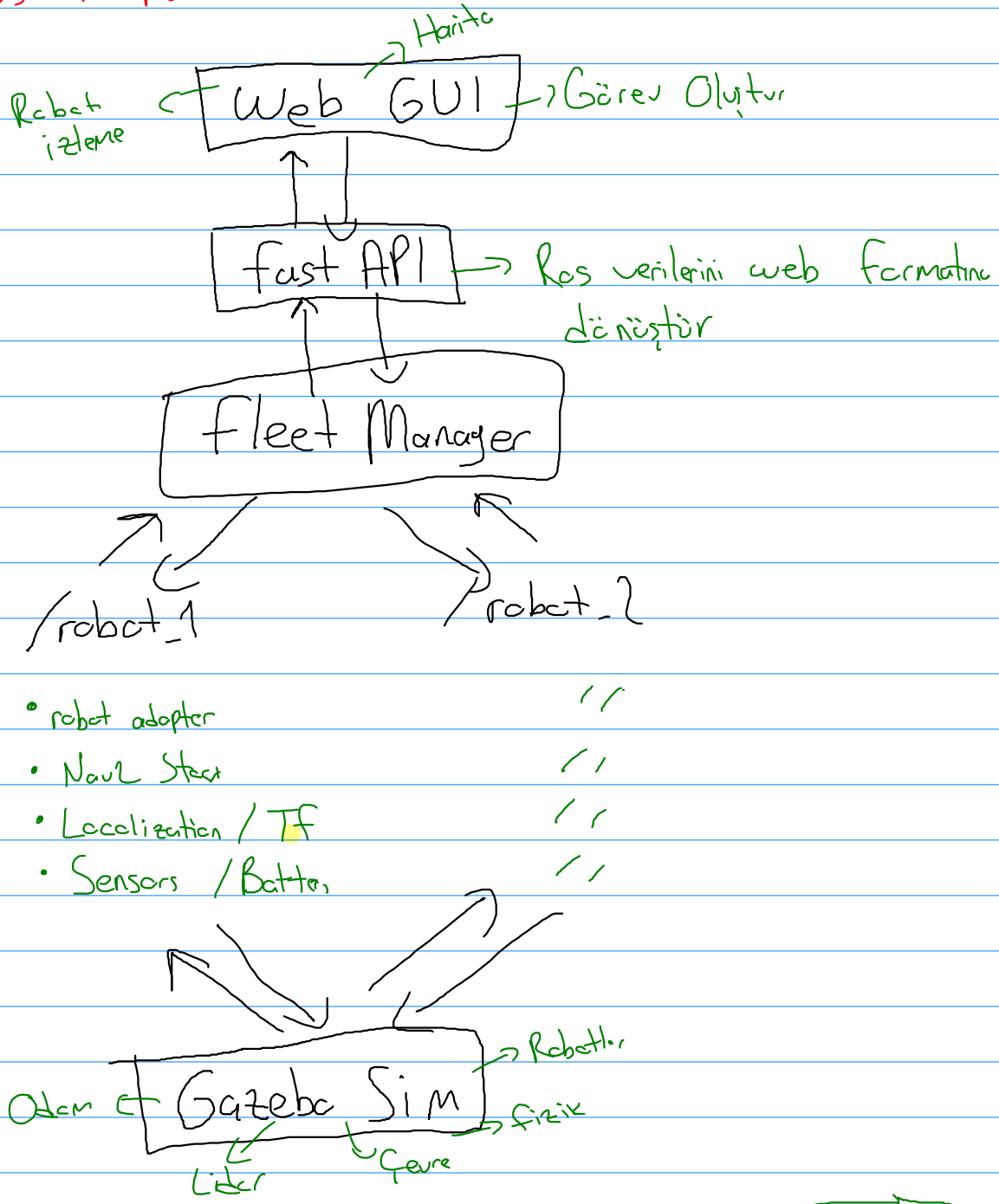
⚠ Gazebo görüntüsü yayınlanmıyacak. Robotun konumu, harita ve rotaları ROS2 topic alıp GUI çizilir.



Kullanılacak Teknoloji Seti

Simülasyon → Gazebo Sim (headless)
Robot haberleşme → Ros 2
Navigasyon → Nav2
Filo Yönetimi → Python
Backend → Fast API
Gerçek zamanlı iletişim → WebSocket
FrontEnd → React + TypeScript
2D Harita Çizimi → React Konva
Veri Saklama → SQL Lite

Ros Mimarisi



Ros2 Paket Yapısı

```
ros2_fleet_ws/
└ src/
    ├ fleet_interfaces → özel msg, srv, action
    ├ fleet_bringup → Launch dosyaları ve parametreler.
    ├ fleet_manager → Görev kuyruğu, atama algoritmaları, durum makinesi
    ├ fleet_robot_adapter → Robot başına Nav2 ile iletişim, durum raporlama
    ├ fleet_monitor → Robotların sağlık durumunu izleme
    ├ fleet_map → Ortak harita ve alma/bırakma istasyonları
    ├ fleet_simulation → Gazebo world, robot spawn ve simülasyon köprüsü
    └ fleet_web_bridge → Ros2 ile fastAPI iletişim
```

/map # Ortak harita

/map_server # Merkezi harita sunucu

/fleet/
- manager
- monitor
- tasks
- robots

/robot_1/
- scan
- odom
- cmd_vel
- battery_state
- navigate_to_pose
- robot_adapter

1) fleet Manager görevi atar. → task_42 → robot_1

2) Robot Adapter alma noktasına gider. → /robot_1/navigate_to_pose action'ına A konumunu gönderir.

3) Yük alma aşaması → Yük alma onayı alır.

4) Bırakma noktasına gider → Aynı Nav2 action'ına B konumunu gönderir.

5) Görevi tamamlar → fleet Manager'a sonucu gönderir robot IDLE olur.

Robot Durumları

OFFLINE
↓
INITIALIZING
↓
IDLE ⟷ CHARGING
↓
BUSY
↓
IDLE

hata durumu → ERROR / ESTOP

Görev Durumları

Queued
↓
Assigned
↓
To_Pickup
↓
Picking_up
↓
To_Dropoff
↓
Completed

- FAILED
- CANCELLED
- Retry_Wait

Robot Seçimi

TurtleBot 3 Burger

Harita Seçimi

- Hazır harita + AMCL

Geliştirme Ortamı

Macbook + Docker

- Ubuntu 22.04 Arm64
- Ros2 Jazzy
- Gazebo Headless
- Nav2

Nav2 Mimarisi

fleet Manager

Robot 1
- Nav2 Stack
- AMCL + Costmaps
- Planner + Controller

Robot 2
- "
- "
- "

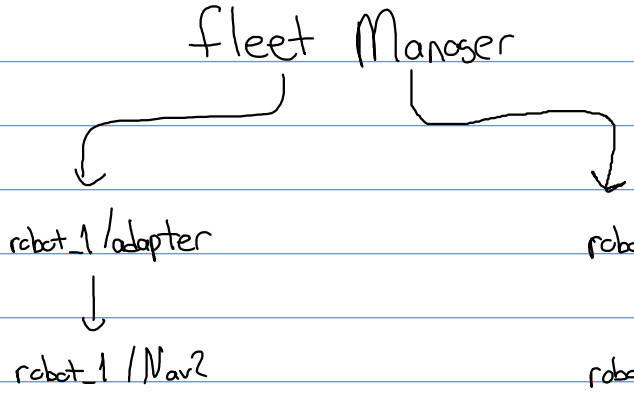
fleet Manager ile robotların erişimi

Robot Adapter ile

↳ Her robotta bir robot_adapter node bulunur.

fleet manager görevleri adapter üzerinden gönderir.

Adapter Nav2'yi yönetir, robotun durumunu takip eder ve fleet Manager'a bildirir.



Robotların Görevleri nasıl tanımlanacak

↳ Tek action ile alma-bırakma görevi

ExecuteTask.action

```
#Goal
string task_id
geometry_msg/PoseStamped    pickup
geometry_msgs/PoseStamped   dropoff
---
#Result
bool success
string messsge
---
#feedback
string stage
float32 progress
```

Robot Durumları Yayınlama Mimarisi

↳ Her robot tek bir birleşik durum mesajı yayınlar

/robot_1/fleet_status

/robot_2/fleet_status

RobotStatus.msg

```
std_msgs/Header header
string robot_id
uint8 state        → CHARGING → IDLE → BUSY
geometry_msgs/Pose pose
float32 battery_percent
string current_task_id
bool navigation_ready
```

Görev Yönetim Mimarisi

FIFO + Priority
- LOW
- MEDIUM
- HIGH

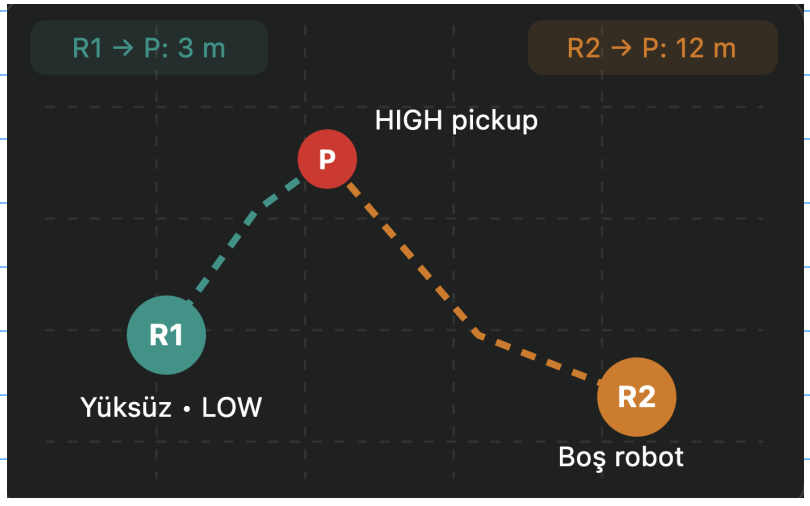
} Mimari Conditional Preemption

HIGH durum kesme

Robotun Aşaması	Yük Durumu	HIGH için kesilebilir mi
IDLE	Yüksüz	Zaten müsait
TO_PICKUP	Yüksüz	Evet
PICKING_UP	Belirsiz	Hayır
TO_DROPOFF	Yüklü	Hayır
CHARGING	Yüksüz	? Şarj Politikasına bağlı
ERROR	Belirsiz	Hayır.

Öncelik Kuralları

- HIGH, MEDIUM ve LOW görevlerinin önüne geçebilir.
- MEDIUM, LOW görevleri kesilebilir ama HIGH görevleri kesemez
- LOW hiçbir aktif görevi kesemez
- Eşit öncelikli görevler birbirini kesemez
- Güvenlik sebebi ile acil durdurma, bu kurallardan bağımsızdır.



Robot 1: LOW görevinde yüksüz, yeni pickup a 3m

Robot 2: IDLE, yeni pickup 12 m

Yeni Görev HIGH

Robot 1 → HIGH görevine geçer

~~Robot 2 → Robot 1'in eski LOW görevini alır.~~

$C_{mevcut} = C(R_1, Low) + C(R_2, High)$

$C_{yeni} = C(R_1, High) + C(R_2, Low)$

→ Maliyet yalnızca mesafeyi değil, eski görevin gecikmesini ve görev kesme cezasında içerebilir.

→ Nearest algoritmasının genişletilmiş versiyonu olur. Buna **Reallocation Policy** denir

Priority-Aware Dynamic Task Reallocation

1) **Güvenlik:** Yüklü, yükleme yapan veya boşaltma yapan robotların görevleri öncelik nedeni ile kesilmez

2) **Öncelik:** HIGH görevler MEDIUM ve LOW görevlerden önce değerlendirilir. Ancak daha düşük öncelikli görevler sistemden silinmez

3) **Verimlilik:** Görevlerin yeniden dağıtılması, gecikme ve kesinti maliyetleri hesaba katıldığında anlamlı bir fayda sağlamalıdır.

Task Manager

↓

Scheduler

↓

Eligibility filter

↓

Reallocation Planner

↓

Assignment Executor

↓

Robot 1 Adapter | Robot 2 Adapter

Cost Calculator

$C = w_1 D + w_2 T + w_3 E + w_4 P$

D: Rota mesafesi

T: Tahmini görev süresi

E: Tahmini enerji tüketimi

P: Görev kesme ve gecikme cezası

w: Normalize edilmiş maliyet ağırlıkları

Reallocation Executor

Görevleri değiştirecek

1) Yeni görev için de

2) İlgili robotlar ve görevler geçici olarak REASSIGNED durumuna alır.

3) Robot 1'in adapter'ına görevi iptal isteği gönderir.

4) Robot 1 durduğunu ve henüz yük almadığını doğrular.

5) Fleet Manager eski LOW görevini tekrar atanabilir hale getirir.

6) Robot 1'e HIGH, Robot 2'ye LOW görevi gönderilir ve her iki action'ın kabülü izlenir.

Scheduler iki plan düşünecek

Yeni bir HIGH görev geldiğinde:

$$I = \frac{C_{mevcut} - C_{yeni}}{C_{mevcut}} \times 100$$

Verim %15 aşağı ise swap işlemi yapmaz

Yeni görev	Davranış
LOW	Normal kuyruğa girer, boş robot bekler
MEDIUM	LOW'dan önce değerlendirilir, boş robot bekler
HIGH	Boş ve görevi kesilebilir robotlar değerlendirilir; verimli olan atama seçilir

HIGH için algoritmamız:

1. Yeni HIGH görev gelir.
2. Fleet Manager boş robotları ve `TO_PICKUP` durumundaki yüksüz robotları değerlendirir.
3. İki atama planının maliyetini karşılaştırır.
4. Görev değişimi **en az %15 verimlilik iyileşmesi** sağlıyorsa uygulanır.
5. Eski görev boş robota aktarılır veya kuyruğa döner.
6. %15 iyileşme yoksa mevcut görev dağılımı korunur.

Kesin güvenlik kuralları

- `TO_PICKUP` ve yüksüz → Görev kesilebilir.
- `PICKING_UP` → Görev kesilemez.
- `TO_DROPOFF` ve yüklü → Görev kesilemez.
- `DROPPING_OFF` → Görev kesilemez.
- `CHARGING` → Yeterli enerji varsa yeni görev alabilir.

Önemli: HIGH bile olsa %15 eşiği aşılmadan mevcut görev kesilmeyecek. Ayrı süre istisnaları veya öncelik katsayıları eklemiyoruz.

Böylece mimarimiz basit kalıyor:

Priority Queue → HIGH ise Reallocation Check → Robot Assignment → ExecuteTask

Verimlilik Hesabı

8.8. Verimlilik nasıl hesaplanacak?

Benim önerim ilk sürümde **Nav2'nin hesapladığı rota mesafesini** kullanmamız.

Bunun için `ComputePathToPose` action'ından rota alıp yol üzerindeki noktaların mesafelerini toplarız.

Her görev için:

$$D(R,T) = D_{\text{robot}\to\text{pickup}} + D_{\text{pickup}\to\text{dropoff}}$$

Böylece robotun sadece pickup'a yakınlığına değil, görevi tamamlamak için gereken toplam mesafeye bakabiliriz.

Örnek: HIGH görev geldi

Robot 1
LOW görevinde, yüksüz

Robot 2
IDLE

Nav2'den hesaplanan örnek toplam mesafeler

Atama	Mesafe
R1 → LOW	15 m
R2 → HIGH	12 m
R1 → HIGH	3 m
R2 → LOW	8 m

Mevcut dağılım:

$$C_{\text{mevcut}} = 15 + 12 = 27 \text{ m}$$

Alternatif dağılım:

$$C_{\text{yeni}} = 3 + 8 = 11 \text{ m}$$

Verimlilik iyileşmesi:

$$I = \frac{27 - 11}{27} \times 100 = \boxed{59.26\%}$$

%59,26 iyileşme, belirlediğimiz %15 eşiğinin üzerinde olduğu için R1'in LOW görevi gü
kesilebilir. R1 HIGH görevi alırken R2 LOW görevini devralabilir.